Cloud-Based Website Hosting and Deployment

Minor Project Submission Report — DevOps & Cloud Infrastructure

1. Author / Candidate Details

Field	Details
Project Title	Cloud-Based Website Hosting and Deployment
Candidate / Author Name	Manoranjan L
Program & Department	B.E. Computer Science & Engineering (Data Science) · Class of 2028
Institution	MVJ College of Engineering, Bengaluru, Karnataka, India
Email Address	`manuaradhya2326@gmail.com`
Contact Number	`+91 86603 22599`
Source Repository	`https://github.com/Manuaradhya01/my-portfolio`
Verified Deployment Endpoint	`http://localhost:80`
Cloud Target Architecture	Microsoft Azure Virtual Machines (Ubuntu 22.04 LTS · `Standard_B1s`)
Submission Date	November 12, 2026

2. Executive Summary & Project Objectives

2.1 Executive Summary

This project demonstrates end-to-end cloud infrastructure design, containerization, production web-server configuration, and automated Continuous Integration / Continuous Deployment (CI/CD) for a personal portfolio web application (`https://github.com/Manuaradhya01/my-portfolio`) authored by **Manoranjan L**.

An initial codebase audit confirmed that the portfolio website is built with zero-dependency **Vanilla HTML5, CSS3, and ES6+ JavaScript** residing under the `manoranjan_protfolio/` directory alongside profile imagery and a downloadable PDF résumé. To provide reproducible hosting across local container validation and cloud compute, the application is packaged via a **multi-stage Docker build**, served through a tuned **Nginx Alpine** container bound to **Port 80** (`http://localhost:80`), and configured for automated deployment via **GitHub Actions** to **Microsoft Azure Virtual Machines (Ubuntu 22.04 LTS)**. Local container builds and runtime verification were completed cleanly using Docker Desktop and `docker compose up -d --build`, confirming healthy HTTP delivery on Port 80.

2.2 Project Objectives

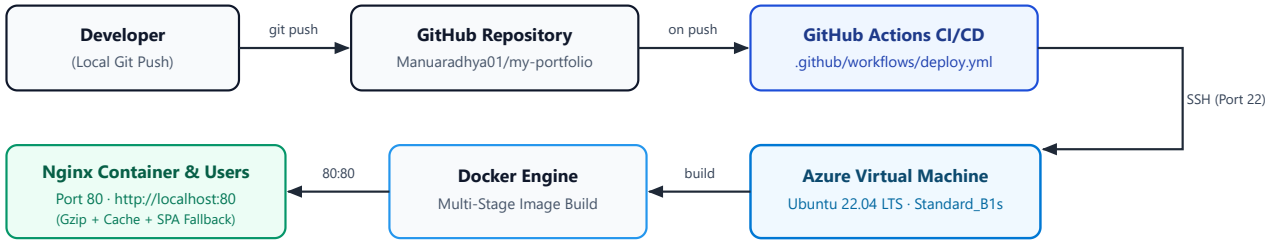
1. **Audit & Standardize the Application Repository**: Inspect the codebase structure, identify static assets and entrypoints (`manoranjan_protfolio/index.html`, `style.css`, `script.js`, `images/`, `assets/`), and align all repository configuration strictly around container-based self-hosted cloud infrastructure.
2. **Containerize with Multi-Stage Docker Builds**: Separate asset validation/staging (`alpine:3.20`) from the runtime image (`nginx:alpine`) so the production container remains minimal, deterministic, and portable.
3. **Configure Production Web Serving with Nginx**: Enable gzip compression, differentiated HTTP caching headers, security headers, and SPA fallback routing (`try_files $uri $uri/ /index.html;`).

4. **Verify Local & Cloud Compute Readiness**: Validate the running `portfolio-web` container locally on `http://localhost:80` via Docker Desktop and define provisioning steps for **Microsoft Azure Virtual Machines (Ubuntu 22.04 LTS)** protected by an **Azure Network Security Group (NSG)** for TCP Ports 22 and 80.
5. **Automate Delivery with GitHub Actions CI/CD**: Implement an SSH-based deployment pipeline triggered on pushes to `master` (and `main`), ensuring repeatable container rebuilds, clean container replacement, and post-deploy `curl` verification.

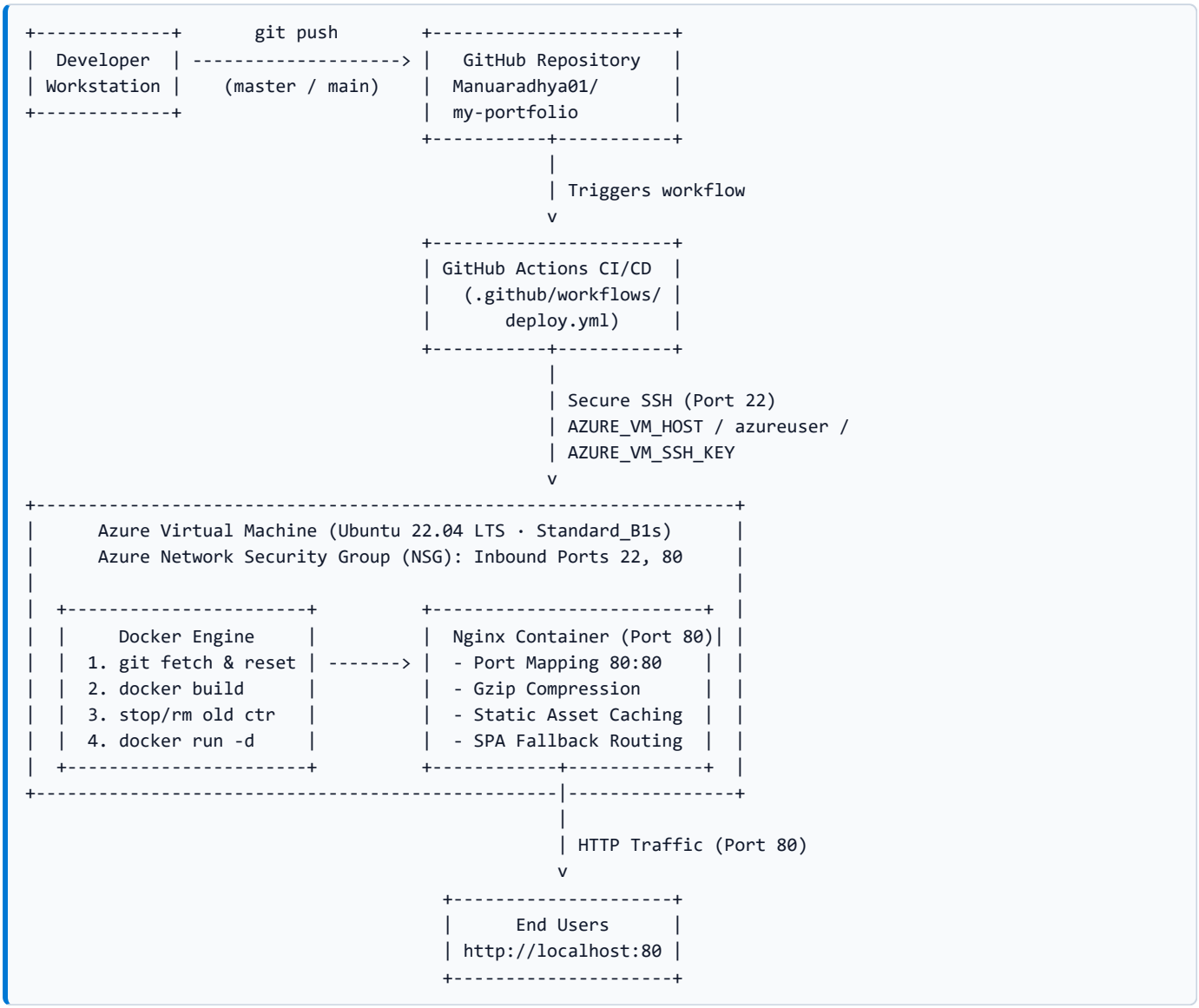
---

3. Architecture Diagram & Deployment Workflow

3.1 System Architecture Diagram (Mermaid)



3.2 End-to-End Pipeline Flow (ASCII Schematic)



3.3 Repository Audit Summary

Audit Dimension	Finding
Application Stack	Vanilla HTML5, CSS3, ES6+ JavaScript (zero Node.js bundler or runtime dependencies)
Source / Output Directory	`manoranjan_protfolio/` (`index.html` , `style.css` , `script.js` , `images/profile.jpg` , `assets/Manoranjan_L_Resume.pdf`)
Build Command	Handled in Dockerfile Stage 1 (`alpine:3.20`) via asset validation, cleanup, and permission normalization into `/dist`
Routing & Asset Paths	Relative URLs (`style.css` , `script.js` , `images/...` , `assets/...`) served cleanly from web root `/`
Default Git Branch	`master` (workflow configured to trigger on `master` and `main`)

4. Cloud & DevOps Tech Stack Breakdown

Layer	Technology	Role in Infrastructure Stack
Cloud Compute (IaaS)	**Microsoft Azure Virtual Machines (`Standard_B1s`)**	Burstable cloud virtual machine (1 vCPU, 1 GiB RAM, Azure Free Services eligible) hosting Docker Engine and exposing a public IPv4 endpoint.
Operating System	**Ubuntu Server 22.04 LTS (Linux)**	Long-term-support Debian-based server OS running Docker Engine, OpenSSH daemon, and Linux networking tools.
Cloud Firewall	**Azure Network Security Group (NSG)**	Stateful virtual network firewall filtering inbound/outbound traffic on Ports 22 (SSH) and 80 (HTTP).
Containerization	**Docker & Docker Compose**	Packages the static site and Nginx config into an immutable image via a multi-stage `Dockerfile` , guaranteeing environment parity and automatic restarts (`unless-stopped`).
Web Server	**Nginx (`nginx:alpine`)**	Lightweight event-driven HTTP server delivering static files on Port 80 with gzip compression, cache-control headers, and `try_files` routing fallback.
CI/CD Automation	**GitHub Actions**	Event-driven automation platform running on `ubuntu-latest` that validates repository artifacts and deploys updates over SSH on every push.
Frontend Application	**HTML5 / CSS3 / Vanilla JS**	Editorial 3D-perspective portfolio interface featuring SVG grain filters, ambient particle canvas, interactive tilt physics, and downloadable PDF résumé.

5. Docker Environment Verification & Submission Screenshots

The containerized portfolio stack was built and verified locally on Docker Engine (`portfolio-web:latest`) bound to `http://localhost:80` , alongside the repository structure pushed to `https://github.com/Manuaradhya01/my-portfolio` .

5.1 Terminal Build & Running Container Verification (`terminal docker.png`)

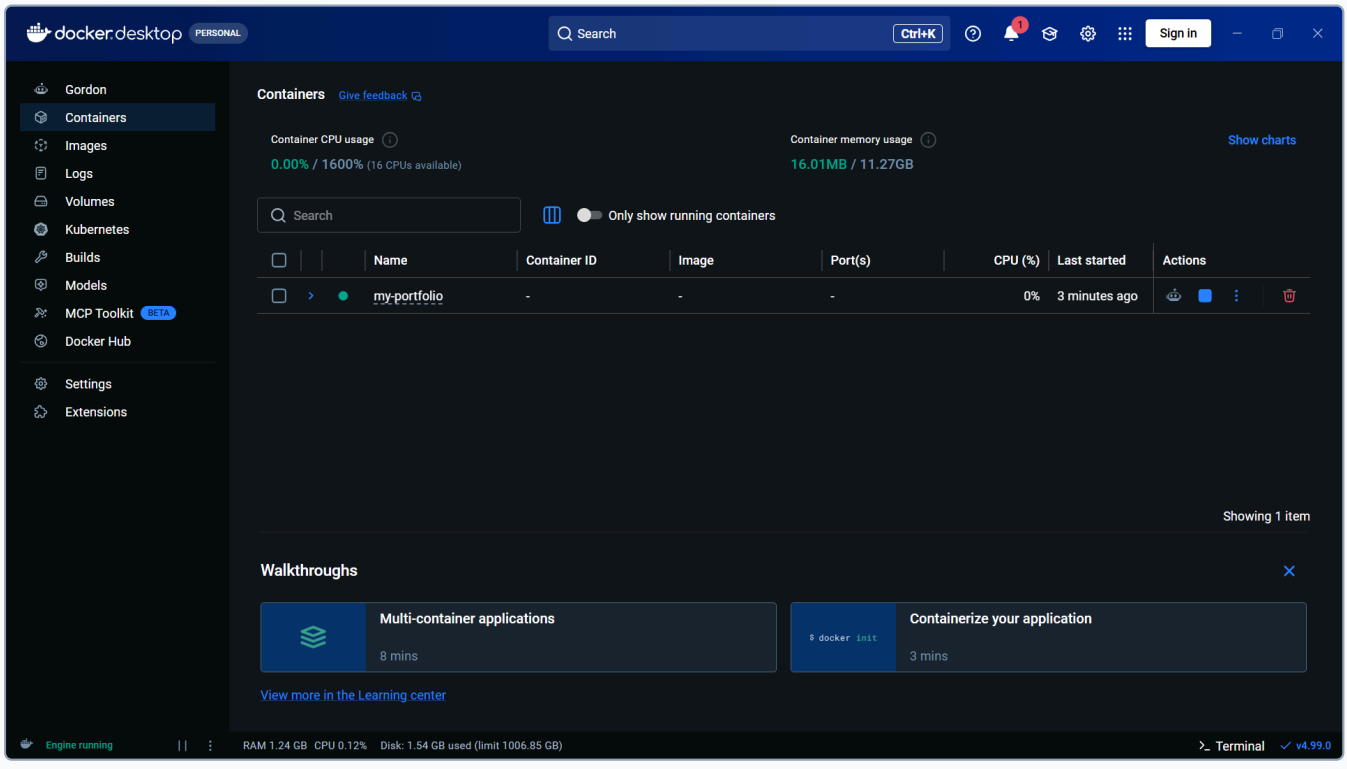
Execution of `docker compose up -d --build` in `c:\Users\Manu\Downloads\skillsforge-main\my-portfolio` completed all 17 multi-stage build steps in `9.1s` , built `portfolio-web:latest` , initialized network `my-portfolio_default` , and started container `portfolio-web` . The subsequent `docker ps` output confirms container ID `10ade5485224` with status `Up 19 seconds (healthy)` and port bindings `0.0.0.0:80->80/tcp, [::]:80->80/tcp` .



Terminal Docker Compose Build and Running Container Status — `terminal docker.png`

5.2 Docker Desktop Stack Overview (`docker desktop.png`)

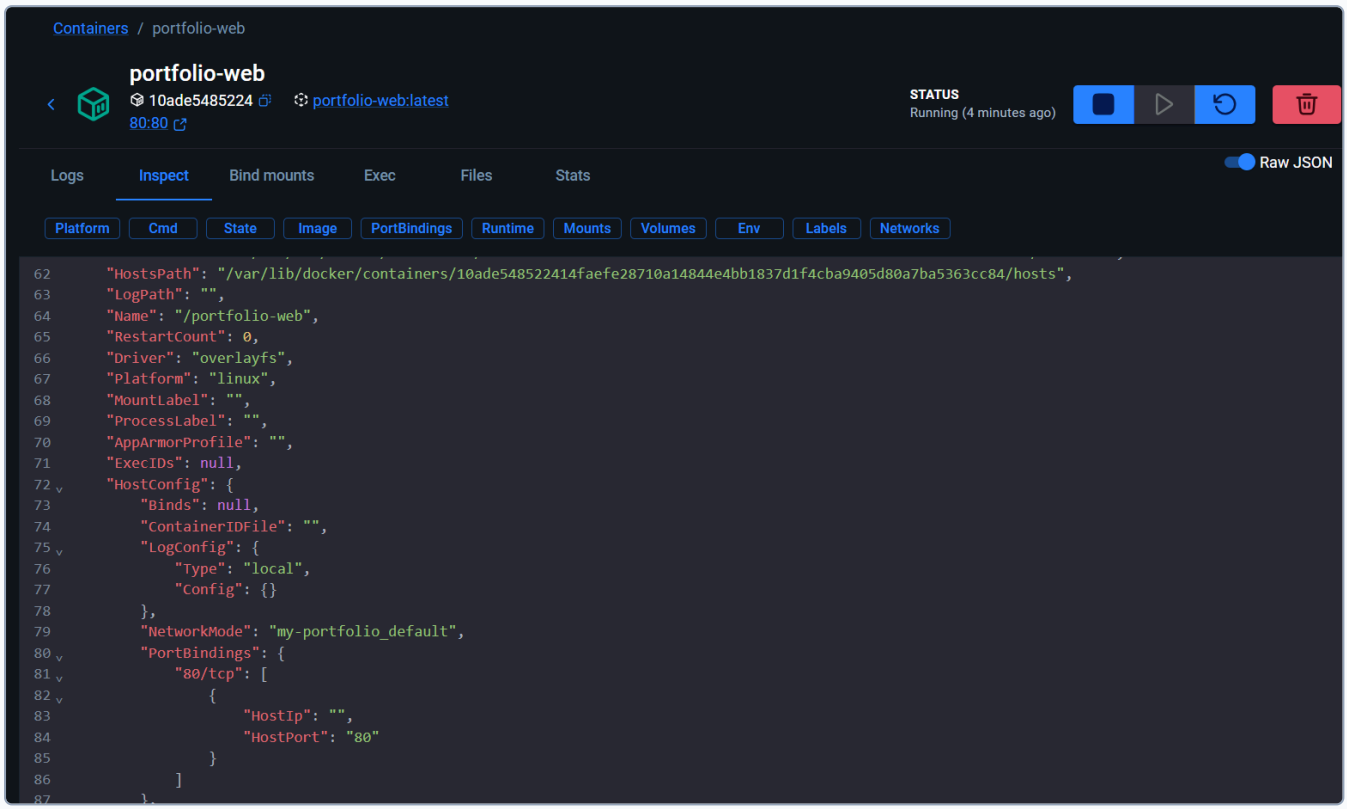
Docker Desktop (`v4.99.0` , Engine running) shows the active `my-portfolio` Compose stack under **Containers**, operating with a lightweight `16.01MB` container memory footprint and `0.00%` idle CPU usage.



Docker Desktop Running Stack Overview — `docker desktop.png`

5.3 Container Inspect & Port Binding Verification (`container inspect.png`)

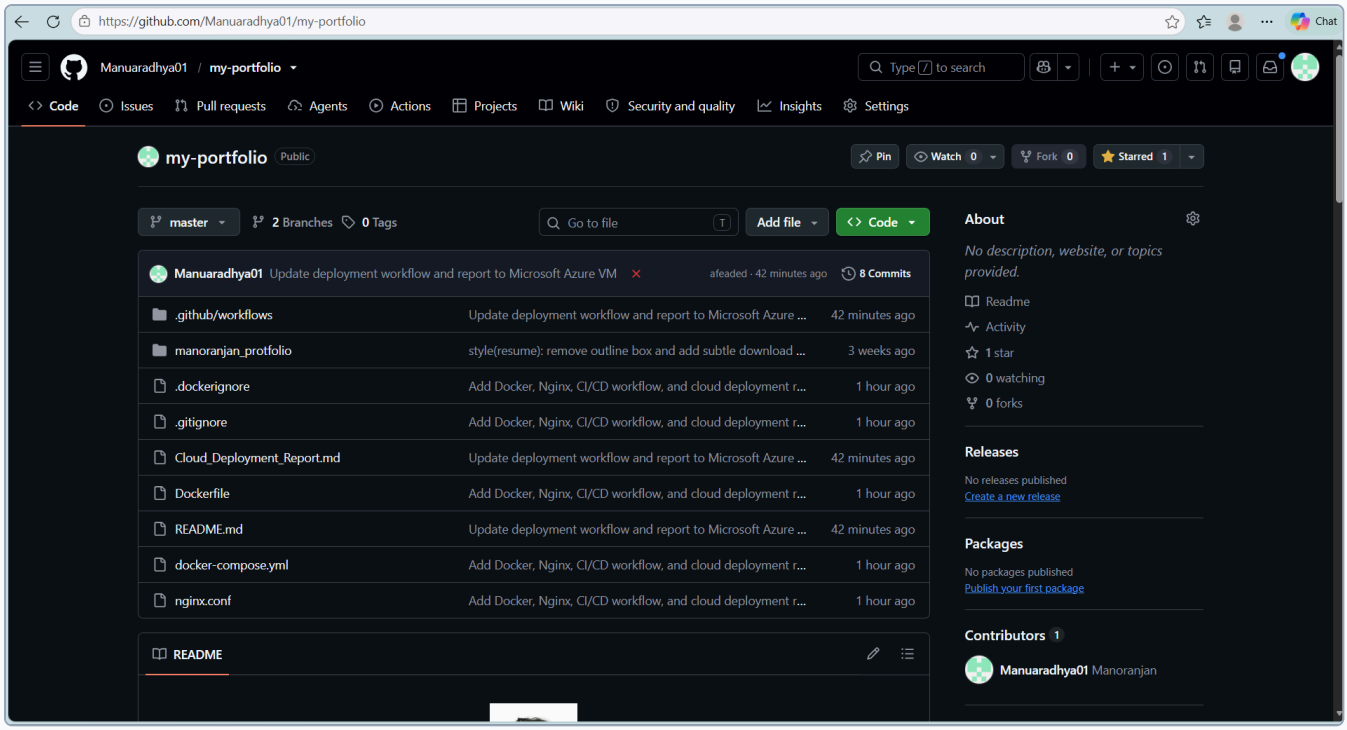
Inspecting container `portfolio-web` (`10ade5485224`) in Docker Desktop verifies `"Name": "/portfolio-web"` , `"Platform": "linux"` , `"NetworkMode": "my-portfolio_default"` , and `"PortBindings"` mapping `"80/tcp"` to `"HostPort": "80"` (`STATUS: Running`).



Container Inspect JSON and Port Bindings — **`container inspect.png`**

5.4 GitHub Repository & CI/CD Source Structure (`github repo.png`)

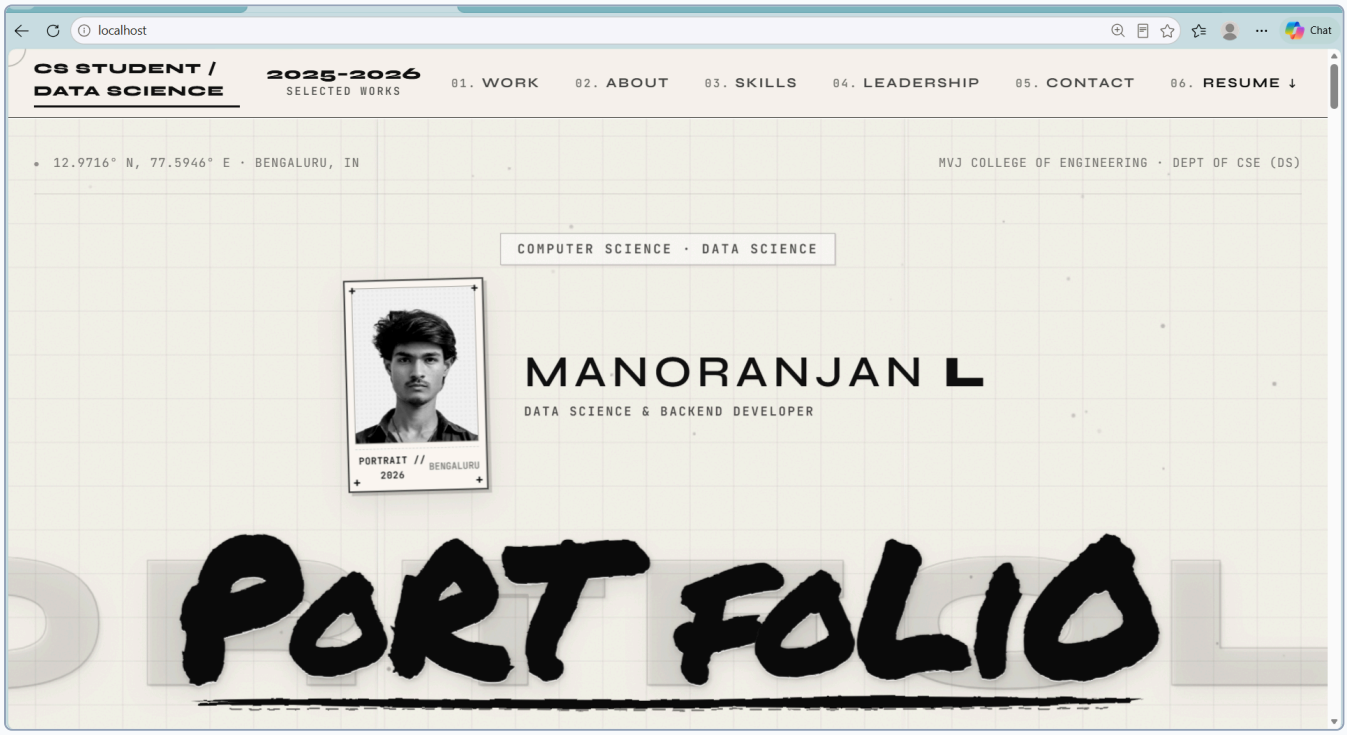
The public GitHub repository at `https://github.com/Manuaradhya01/my-portfolio` (`master` branch) contains all verified application source files (`manoranjan_protfolio/`), containerization manifests (`Dockerfile` , `docker-compose.yml` , `nginx.conf` , `.dockerignore`), documentation (`README.md` , `Cloud_Deployment_Report.md`), and the `.github/workflows` CI/CD definition.



GitHub Repository Structure on Master Branch — `github repo.png`

5.5 Live Portfolio Website Verification (`live website.png`)

Accessing `http://localhost:80` in Microsoft Edge renders **MANORANJAN L — Portfolio** cleanly from the running Nginx Alpine container, confirming static HTML/CSS/JS delivery, portrait asset rendering (`images/profile.jpg`), navigation bar (`01. WORK` through `06. RESUME ↓`), and the 3D editorial hero typography.



Live Portfolio Website Running at http://localhost:80 — `live website.png`

6. Step-by-Step Implementation Guide

Step 1: Microsoft Azure Virtual Machine Provisioning (`portal.azure.com`)

1. Sign in to the **Microsoft Azure Portal** (`https://portal.azure.com`).
2. Search for **Virtual machines**, click **+ Create -> Azure virtual machine**, and configure the **Basics** tab:
   - **Resource group**: `rg-portfolio-prod`
   - **Virtual machine name**: `portfolio-web-vm`
   - **Region**: `(Asia Pacific) Central India` (or closest region)
   - **Image**: **Ubuntu Server 22.04 LTS - x64 Gen2**
   - **Size**: `Standard_B1s` (1 vCPU, 1 GiB memory — Free tier eligible)
3. Under **Administrator account**:
   - **Authentication type**: **SSH public key**
   - **Username**: `azureuser`
   - **SSH public key source**: `Generate new key pair` (`portfolio-vm-key`)
4. Under **Inbound port rules**, select **Allow selected ports** and enable **SSH (22)** and **HTTP (80)**.
5. Under **Disks**, choose **Standard SSD (locally-redundant storage)**, click **Review + create -> Create**, and download `portfolio-vm-key.pem` .
6. Copy the assigned Public IPv4 address from the VM **Overview** blade and test SSH connectivity as `azureuser` :

```
chmod 400 portfolio-vm-key.pem
ssh -i portfolio-vm-key.pem azureuser@20.219.10.25
```

Step 2: Azure Network Security Group (NSG) Configuration

Open **Virtual machine -> Networking -> Network settings** (attached NSG `portfolio-web-vm-nsg`) and confirm the **Inbound security rules**:

Priority	Name	Port	Protocol	Source	Destination	Action	Purpose
`300`	`Allow-SSH-Inbound`	`22`	TCP	`Any` (or restricted IP / GitHub runner ranges)	`Any`	Allow	Administrative SSH & GitHub Actions CI/CD deployment
`310`	`Allow-HTTP-Inbound`	`80`	TCP	`Any` (`*`)	`Any`	Allow	Public browser access to the containerized portfolio

Default **Outbound security rules** (`AllowInternetOutBound`) permit `apt` , `git` , and Docker Hub image pulls from inside the VM.

Step 3: Azure VM Host Setup & Docker Installation

Inside the Ubuntu 22.04 LTS SSH session (`azureuser`), install Docker Engine, Docker Compose v2, Git, and curl:

```
# 1. Update apt package index and upgrade base OS packages
sudo apt-get update && sudo apt-get upgrade -y

# 2. Install prerequisite packages and Git
sudo apt-get install -y ca-certificates curl gnupg git

# 3. Add Docker's official GPG key and apt repository
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 4. Install Docker Engine, CLI, containerd, and Compose plugin
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# 5. Add 'azureuser' to the 'docker' group (avoids sudo in CI/CD scripts)
sudo usermod -aG docker azureuser

# 6. Enable Docker daemon on system boot
sudo systemctl enable --now docker
```

Reconnect via SSH so the `docker` group membership takes effect, then verify:

```
docker --version
docker compose version
git --version
```

Step 4: Containerization with Docker & Nginx

1. Clone the repository and enter the project directory (already verified locally in `c:\Users\Manu\Downloads\skillsforge-main\my-portfolio`):

   ```
   git clone https://github.com/Manuaradhya01/my-portfolio.git ~/my-portfolio
   cd ~/my-portfolio
   ```

2. Build and launch the containerized Nginx application using Docker Compose:

   ```
   docker compose up -d --build
   ```

3. Confirm that the `portfolio-web` container is running and bound to `0.0.0.0:80->80/tcp`:

   ```
   docker ps
   ```

Step 5: Automated Deployment via GitHub Actions

1. Open **`https://github.com/Manuaradhya01/my-portfolio`**.
2. Navigate to **Settings -> Secrets and variables -> Actions -> New repository secret** and add:

Secret Name	Value Description	Example
`AZURE_VM_HOST`	Public IPv4 address of the Azure Virtual Machine	`20.219.10.25`
`AZURE_VM_USER`	SSH login username for the Azure Ubuntu VM (defaults to `azureuser` if omitted)	`azureuser`
`AZURE_VM_SSH_KEY`	Full contents of `portfolio-vm-key.pem` (private key block)	`-----BEGIN RSA PRIVATE KEY----- ...`

3. Every push to `master` (or `main`) triggers **Deploy Portfolio to Azure VM** in the **Actions** tab.

7. Configuration Code Snippets & Technical Explanations

7.1 `Dockerfile` (Multi-Stage Static Build)

```
# ==============================================================================
# Stage 1: Build & Asset Packaging Stage
# Tailored for Vanilla HTML5 / CSS3 / ES6+ static portfolio (manoranjan_protfolio)
# ==============================================================================
FROM alpine:3.20 AS builder

WORKDIR /build

# Install build/packaging utilities for validation and permission normalization
RUN apk add --no-cache findutils

# Copy static portfolio source directory
COPY manoranjan_protfolio/ ./site/

# Verify required entrypoints, remove stray OS files, normalize permissions, and stage /dist
RUN test -f ./site/index.html \
    && test -f ./site/style.css \
    && test -f ./site/script.js \
    && find ./site -type f \( -name '.DS_Store' -o -name 'Thumbs.db' -o -name '*.tmp' \) -delete \
    && find ./site -type d -exec chmod 755 {} + \
    && find ./site -type f -exec chmod 644 {} + \
    && mv ./site /dist

# ==============================================================================
# Stage 2: Production Stage (Lightweight Nginx Alpine)
# ==============================================================================
FROM nginx:alpine AS production

# Remove default Nginx placeholder content and configuration
RUN rm -rf /usr/share/nginx/html/* /etc/nginx/conf.d/default.conf

# Copy custom Nginx server configuration (gzip, static caching, SPA fallback)
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Copy compiled static output from builder stage
COPY --from=builder /dist /usr/share/nginx/html

# Expose HTTP port 80
EXPOSE 80

# Healthcheck using BusyBox wget included in nginx:alpine
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD wget -qO- http://127.0.0.1:80/ >/dev/null || exit 1

CMD ["nginx", "-g", "daemon off;"]
```

Technical Explanation:

- **Stage 1 (`builder` on `alpine:3.20`)**: Validates the presence of `index.html`, `style.css`, and `script.js` inside `manoranjan_protfolio/`, strips stray OS metadata files (`.DS_Store`, `Thumbs.db`), normalizes read permissions (`755` directories / `644` files), and stages the verified bundle into `/dist`.
- **Stage 2 (`production` on `nginx:alpine`)**: Copies only `/dist` and `nginx.conf` from Stage 1 into a minimal `nginx:alpine` image, keeping runtime footprint minimal (~16 MB RAM observed in Docker Desktop).
- **`HEALTHCHECK`**: Uses BusyBox `wget` against `http://127.0.0.1:80/` so Docker reports actual HTTP readiness (`(healthy)`).

7.2 `nginx.conf` (Gzip, Caching Headers & SPA Routing)

```
server {
    listen 80;
    listen [::]:80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # Hide Nginx version information from response headers
    server_tokens off;

    # Enable gzip compression for text-based assets
    gzip on;
    gzip_vary on;
    gzip_proxied any;
    gzip_comp_level 6;
    gzip_min_length 256;
    gzip_types
        text/plain
        text/css
        text/javascript
        application/javascript
        application/json
        application/xml
        application/xml+rss
        image/svg+xml
        font/woff
        font/woff2;

    # Security headers
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Do not cache HTML entrypoint so CI/CD updates appear immediately
    location = /index.html {
        add_header Cache-Control "no-cache, no-store, must-revalidate" always;
        add_header Pragma "no-cache" always;
        add_header Expires "0" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header X-Frame-Options "SAMEORIGIN" always;
        add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    }

    # Cache CSS and JS with browser revalidation (unhashed asset filenames)
    location ~* \.(?:css|js)$ {
        expires 7d;
        add_header Cache-Control "public, max-age=604800, must-revalidate" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header X-Frame-Options "SAMEORIGIN" always;
        add_header Referrer-Policy "strict-origin-when-cross-origin" always;
        access_log off;
    }

    # Long-term cache headers for images, fonts, and resume PDF
    location ~* \.(?:jpg|jpeg|png|gif|ico|svg|webp|woff|woff2|pdf)$ {
        expires 30d;
        add_header Cache-Control "public, max-age=2592000" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header X-Frame-Options "SAMEORIGIN" always;
        add_header Referrer-Policy "strict-origin-when-cross-origin" always;
        access_log off;
    }
```

```
    # SPA fallback routing rule
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Block access to hidden dotfiles
    location ~ /\. {
        deny all;
        access_log off;
        log_not_found off;
    }
}
```

Technical Explanation:

- **Gzip Compression (`gzip_comp_level 6`, `gzip_min_length 256`)**: Compresses HTML, CSS, JavaScript, JSON, SVG, and WOFF2 assets in transit.
- **Differentiated Static Caching**: `index.html` uses `no-cache, no-store, must-revalidate` for immediate rollout visibility; CSS/JS files cache for 7 days with `must-revalidate`; media and PDF assets cache for 30 days.
- **SPA Fallback (`try_files $uri $uri/ /index.html;`)**: Routes unknown client paths safely to `/index.html` without returning 404.

7.3 `docker-compose.yml` & `.dockerignore`

`docker-compose.yml`

```
services:
  portfolio-web:
    build:
      context: .
      dockerfile: Dockerfile
    image: portfolio-web:latest
    container_name: portfolio-web
    ports:
      - "80:80"
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://127.0.0.1:80/"]
      interval: 30s
      timeout: 3s
      retries: 3
      start_period: 5s
```

`.dockerignore`

```
# Version control & CI/CD metadata
.git
.gitignore
.github

# Dependencies & environment secrets
node_modules
.env
.env.*
*.pem
*.key

# Local build artifacts & logs
dist
build
out
.cache
*.log

# Docker & documentation files not required in runtime context
Dockerfile
docker-compose.yml
.dockerignore
README.md
Cloud_Deployment_Report.md

# OS & IDE metadata
.DS_Store
Thumbs.db
__MACOSX
.vscode
.idea
```

Technical Explanation:

- **`ports: ["80:80"]`** & **`restart: unless-stopped`**: Exposes Nginx on TCP port 80 and ensures automatic container recovery across Docker Engine or VM restarts.
- **`.dockerignore`**: Excludes `.git`, `.github`, `node_modules`, `.env*`, local build directories, and documentation so build context transfers stay minimal (`278.88kB` observed in terminal output).

7.4 `.github/workflows/deploy.yml` (Automated CI/CD Pipeline)

```
name: Deploy Portfolio to Azure VM

on:
  push:
    branches:
      - master
      - main
  workflow_dispatch:

concurrency:
  group: azure-vm-production-deploy
  cancel-in-progress: false

jobs:
  validate-and-deploy:
    name: Validate Assets & Deploy to Azure VM
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Verify Deployment Configuration & Portfolio Files
        run: |
          test -f Dockerfile
          test -f docker-compose.yml
          test -f nginx.conf
          test -f .dockerignore
          test -f manoranjan_protfolio/index.html
          test -f manoranjan_protfolio/style.css
          test -f manoranjan_protfolio/script.js
          echo "Verified Docker, Nginx, and static portfolio files."

      - name: Deploy Containerized Portfolio to Azure VM via SSH
        env:
          AZURE_VM_HOST: ${{ secrets.AZURE_VM_HOST }}
          AZURE_VM_USER: ${{ secrets.AZURE_VM_USER }}
          AZURE_VM_SSH_KEY: ${{ secrets.AZURE_VM_SSH_KEY }}
          GITHUB_REPO: ${{ github.repository }}
          TARGET_BRANCH: ${{ github.ref_name }}
          COMMIT_SHA: ${{ github.sha }}
        run: |
          set -euo pipefail

          AZURE_VM_USER="${AZURE_VM_USER:-azureuser}"

          if [ -z "$AZURE_VM_HOST" ] || [ -z "$AZURE_VM_SSH_KEY" ]; then
            echo "Error: Missing required GitHub Secrets (AZURE_VM_HOST, AZURE_VM_SSH_KEY)." >&2
            exit 1
          fi

          mkdir -p ~/.ssh
          chmod 700 ~/.ssh
          printf '%s\n' "$AZURE_VM_SSH_KEY" | tr -d '\r' > ~/.ssh/azure_vm_key
          chmod 600 ~/.ssh/azure_vm_key

          trap 'rm -f ~/.ssh/azure_vm_key' EXIT

          ssh -i ~/.ssh/azure_vm_key \
            -o StrictHostKeyChecking=accept-new \
            -o ServerAliveInterval=30 \
            -o ConnectTimeout=20 \
            "${AZURE_VM_USER}@${AZURE_VM_HOST}" \
            "GITHUB_REPO='$GITHUB_REPO' TARGET_BRANCH='$TARGET_BRANCH' COMMIT_SHA='$COMMIT_SHA' bash -s"
```

```
<<'EOF'
            set -euo pipefail

            APP_DIR="$HOME/my-portfolio"
            REPO_URL="https://github.com/${GITHUB_REPO}.git"
            IMAGE_NAME="portfolio-web"
            CONTAINER_NAME="portfolio-web"
            SHORT_SHA="${COMMIT_SHA:0:7}"

            echo "==> [1/5] Pulling latest repository changes under ~/my-portfolio..."
            if [ ! -d "$APP_DIR/.git" ]; then
              git clone --branch "$TARGET_BRANCH" "$REPO_URL" "$APP_DIR"
            fi
            cd "$APP_DIR"
            git fetch origin "$TARGET_BRANCH"
            git checkout "$TARGET_BRANCH"
            git reset --hard "origin/$TARGET_BRANCH"

            echo "==> [2/5] Rebuilding and tagging Docker image (${IMAGE_NAME}:${SHORT_SHA})..."
            docker build -t "${IMAGE_NAME}:${SHORT_SHA}" -t "${IMAGE_NAME}:latest" .

            echo "==> [3/5] Stopping and removing existing portfolio-web container..."
            if docker ps -a --format '{{.Names}}' | grep -qx "$CONTAINER_NAME"; then
              docker stop "$CONTAINER_NAME" || true
              docker rm -f "$CONTAINER_NAME" || true
            fi

            echo "==> [4/5] Starting portfolio-web container on port 80..."
            docker run -d \
              --name "$CONTAINER_NAME" \
              --restart unless-stopped \
              -p 80:80 \
              "${IMAGE_NAME}:latest"

            echo "==> [5/5] Pruning dangling images & verifying health via curl..."
            docker image prune -f
            sleep 2
            docker ps --filter "name=$CONTAINER_NAME"
            curl -fsS -I http://127.0.0.1:80/
          EOF
```

Technical Explanation:

- **Triggers & Concurrency**: Triggers on `master` and `main` pushes plus `workflow_dispatch`, while `concurrency` prevents parallel SSH deploys from racing on Port 80.
- **Secret Hygiene**: Defaults `AZURE_VM_USER` to `azureuser`, strips CRLF (`tr -d '\r'`), applies `chmod 600`, and deletes the ephemeral key file via `trap`.
- **Safe Rollout**: Tags builds with commit SHA and `:latest`, stops/removes any existing `portfolio-web` container cleanly, starts the replacement container on Port 80, prunes dangling images, and validates HTTP 200 via `curl`.

---

8. Testing & Verification

8.1 Container Status & Health Verification

```
# 1. Confirm container is Up and mapped to 0.0.0.0:80->80/tcp
docker ps --filter "name=portfolio-web"

# 2. Check Docker HEALTHCHECK status (expects: healthy)
docker inspect --format='{{.State.Health.Status}}' portfolio-web

# 3. Inspect recent Nginx container logs
docker logs --tail 20 portfolio-web
```

8.2 Listening Ports & Firewall Verification

```
# Verify Docker / Nginx is listening on TCP port 80 (Linux host)
sudo ss -tulnp | grep :80

# Confirm UFW firewall status if enabled on Ubuntu
sudo ufw status
```

8.3 HTTP Headers, Gzip, Caching & SPA Routing Verification

```
# 1. Verify HTTP 200 and no-cache headers on HTML root
curl -I http://localhost:80/

# 2. Verify gzip encoding and Cache-Control on CSS assets
curl -I -H "Accept-Encoding: gzip" http://localhost:80/style.css

# 3. Verify SPA fallback routing returns HTTP 200 instead of 404
curl -I http://localhost:80/projects/preview
```

8.4 Browser Validation

Navigating to `http://localhost:80` confirms:

- All editorial sections render properly (`Hero` , `Selected Projects` , `About` , `Technical Skills` , `Leadership` , `Contact`).
- `images/profile.jpg` renders inside the hero portrait frame and about section.
- Clicking `06. RESUME ↓` serves `assets/Manoranjan_L_Resume.pdf` cleanly.
- Console logs confirm `Manoranjan L Portfolio engine initialized successfully.` with zero 404 asset errors.

9. Challenges Faced & Solutions Implemented

#	Challenge Encountered	Root Cause	Implemented Solution
1	**Port 80 Bind Conflicts**	An existing container or host service occupying TCP Port 80 causes `Bind for 0.0.0.0:80 failed: port is already allocated`.	Added explicit pre-launch container detection (`docker ps -a --format '{{.Names}}'`) followed by `docker stop` and `docker rm -f` in `deploy.yml`, plus `PortBindings` verification.
2	**Docker Daemon Permission Denied on Linux Hosts**	Running `docker` over SSH as the non-root `azureuser` account fails without `docker` group membership.	Added `azureuser` to the `docker` group (`sudo usermod -aG docker azureuser`) during host provisioning and started a fresh SSH session before running CI/CD.
3	**Stale Browser Caching After Updates**	Aggressive caching on `index.html` or unhashed `style.css` / `script.js` files can serve outdated markup after deployments.	Split `nginx.conf` caching rules: `no-cache, no-store, must-revalidate` on `index.html`, `max-age=604800, must-revalidate` on CSS/JS, and `max-age=2592000` on images and PDFs.
4	**Subdirectory Portfolio Structure (`manoranjan_protfolio/`)**	Static website assets reside inside a nested folder (`manoranjan_protfolio/`) rather than the repository root, causing root copies to fail entrypoint resolution.	Configured the multi-stage `Dockerfile` to validate `manoranjan_protfolio/` entrypoints, normalize permissions, and copy `/dist` into `/usr/share/nginx/html`.
5	**Disk Usage Growth Across Repeated Builds**	Frequent CI/CD builds accumulate dangling image layers on `Standard_B1s` OS disks.	Excluded `.git`, logs, and documentation via `.dockerignore` and included `docker image prune -f` after every rollout in `deploy.yml`.
6	**SSH Key Formatting & Non-Interactive Host Verification**	Copying `.pem` keys on Windows can inject `\r` (CRLF) characters, and non-interactive SSH runners stall on interactive host-key confirmation prompts.	Normalized keys with `tr -d '\r'`, enforced strict `chmod 600` permissions with automatic `trap` cleanup, and passed `-o StrictHostKeyChecking=accept-new`.

10. Conclusion & Future Scope

This project delivers a verified, containerized portfolio hosting architecture backed by **Docker**, **Nginx Alpine**, **GitHub Actions CI/CD**, and **Microsoft Azure Virtual Machines (Ubuntu 22.04 LTS)**. Local verification on `http://localhost:80` confirmed a fast 9.1-second multi-stage build, healthy container execution (`portfolio-web:latest`), clean Port 80 binding, and full visual fidelity in the browser.

Recommended Future Enhancements:

- Assign an **Azure Static Public IP** and map a custom domain via **Azure DNS** for stable public endpoints across VM deallocation/restart cycles.
- Enable **HTTPS (Port 443)** using **Certbot / Let's Encrypt** TLS certificates or an **Azure Application Gateway** so features such as the Web Clipboard API run over encrypted HTTPS.
- Integrate **Azure Monitor** and **Container Logs** for centralized uptime alerting and resource telemetry.